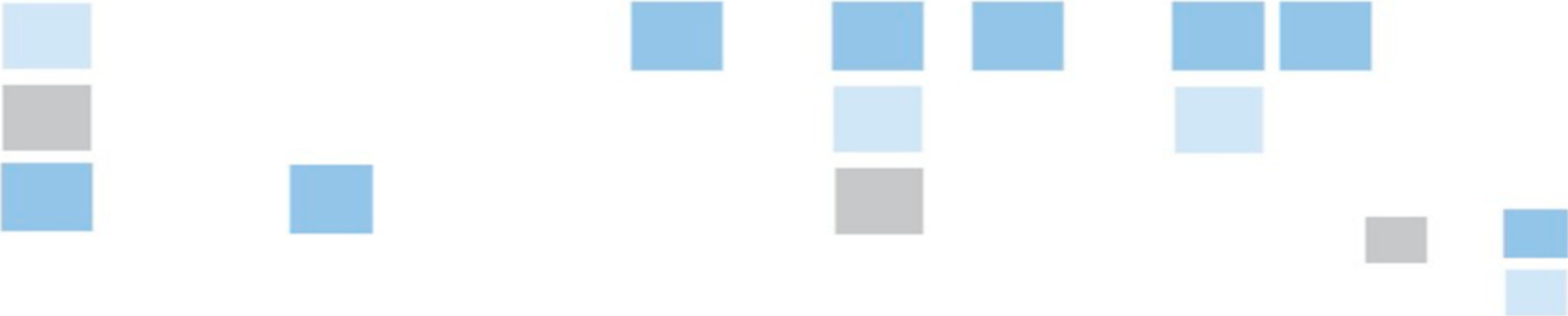


A collection of approximately 15 squares in light blue, medium blue, and grey, scattered across the top half of the slide.

Web Projects I

Introduction to JavaScript: Syntax, Data Types, Variables, and Operators

A collection of approximately 8 squares in light blue, medium blue, and grey, scattered across the bottom half of the slide.

laSalle

UNIVERSITAT RAMON LLULL

Table of Contents

- ❑ Introduction
- ❑ What is JavaScript?
- ❑ Basic Syntax of JavaScript
- ❑ JavaScript Data Types
- ❑ Variables
- ❑ Conditional Statements

Introduction

- ❑ In this session, we'll explore the basics of JavaScript, a powerful scripting language used to create dynamic and interactive experiences on the web.
- ❑ We'll cover the fundamental concepts including syntax, data types, variables, and operators.

What is JavaScript?

- ❑ JavaScript is a high-level, interpreted, single threaded scripting language that conforms to the ECMAScript specification.
- ❑ It enables interactive web pages and is an essential part of web applications. Alongside HTML and CSS, JavaScript is one of the core technologies of the World Wide Web.

What is JavaScript?

- ❑ External scripts are linked to HTML via the `<script src>` element. Javascript scripts can also be included directly within HTML using `<script>` tags.
- ❑ Historically, the script loading was usually done before the closing tag `</body>` to avoid render blocking and ensure that the DOM had loaded.

```
<!--  
  content  
-->  
<script>console.log("hello world")</script>  
<script src="./script.js"></script>  
</body>  
</html>
```

What is JavaScript?

❑ Nowadays, we have properties of the `<script>` that controls how the script should be executed.

- (no attribute)
- defer
- async
- type="module"

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
    <script src="./script.js"></script>
    <script src="./script.js" defer></script>
    <script src="./script.js" async></script>
    <script src="./script.js" type="module"></script>
  </head>
  <body>
    <section>
```

What is JavaScript?

- ❑ The type="module" property enables loading scripts as ES6 modules. This type of scripts are executed in deferred manner, but with support for importing and exporting between modules.
- ❑ Best for writing modern code in Javascript using ES6 modules.

```
// calculator.js
export function sum(a, b) {
  return a + b;
}

export function resta(a, b) {
  return a - b;
}
```

```
<script type="module">
  import { sum } from "./calculator.js";
  console.log(sum(1, 2)); // 3
</script>
```

Basic Syntax of JavaScript

- ❑ JavaScript statements are the commands that tell the browser what to do, typically ending with a semicolon.
- ❑ Use `//` for single-line comments and `/* */` for multi-line comments to annotate code without affecting execution.

```
// This is a single-line comment
/*
This is a multi-line comment
*/
console.log("Hello, world!"); // Outputs to the console
```


JavaScript Data Types

- ❑ **Primitive Data Types:** Includes undefined, null, boolean, number, string, symbol, and bigint.
- ❑ **Complex Data Types:** JavaScript supports objects, including arrays, maps, sets and functions.

```
let name = "John"; // String
let age = 30; // Number
let isActive = true; // Boolean
let user = {firstName: "John", lastName: "Doe"}; // Object
```

Javascript Build-in Objects

❑ Javascript has objects in the global scope.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

❑ Strings

- Are concatenated using the operator '+'.
○ Have methods such as 'toUpperCase()' or 'startsWith()'.

❑ Math

- Has properties such as 'E' or 'PI'.
○ Has methods such as 'random()', 'abs()' or 'floor()'.

Javascript Build-in Objects

Arrays

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

❑ Working with arrays

<code>.flat()</code>	<code>.pop()</code>	<code>.sort()</code>
<code>.forEach()</code>	<code>.push()</code>	<code>.filter()</code>
<code>.join()</code>	<code>.shift()</code>	<code>.find()</code>
<code>.split()</code>	<code>.slice()</code>	<code>.some()</code>
<code>.map()</code>	<code>.reverse()</code>	<code>.every()</code>

Variables

- ❑ **Declaring Variables:** Use `var`, `let`, or `const` to declare variables. `let` and `const` are block-scoped, with `const` defining a constant reference, `var` is function-scoped.
- ❑ Always use `let` or `const` for variable declarations to avoid potential issues with scoping.

```
let message = "Hello, JavaScript!";  
const PI = 3.14159;  
var oldWay = "This is old but still works";
```

Variables

Rules for naming variables:

- ❑ Variable names cannot start with numbers.
- ❑ Variable names are case sensitive, so `myName` and `myname` would be different variables. It is bad practice to create two variables that have the same name using different cases.
- ❑ Variable names cannot be the same as *keywords*. [MDN's keyword documentation](#).

Variables

```
let sum = 10 + 5; // 15
let isAdult = age > 18; // true if age is greater than 18
let counter = 1;
counter++; // Increment counter by 1
```

Operators

□ Types of operators:

- **Arithmetic Operators:** +, -, *, /, %, ++, --
- **Assignment Operators:** =, +=, -=, *=, /=
- **Comparison Operators:** ==, ===, !=, !==, >, <, >=, <=
- **Logical Operators:** &&, ||, !

Conditional Statements: if

- ❑ **If, else if, else:** The `if` keyword followed by a set of parentheses `()` which is followed by a code block, indicated by a set of curly braces `{}`

```
if (logicalCondition1) {  
    console.log('will execute if condition is true');  
} else if (logicalCondition2){  
    console.log('will execute if logicalCondition1 is false' +  
               ' and logicalCondition2 is true');  
} else {  
    console.log('will execute if both conditions are false');  
}
```


Conditional Statements: Ternary Operator

- ❑ The ternary operator is a shorthand conditional operator that evaluates a condition and returns one value or another in a single line.

```
let isNightTime : boolean = true;

if (isNightTime) {
    console.log(...data: 'Turn on the lights!');
} else {
    console.log(...data: 'Turn off the lights!');
}
```

```
let isNightTime : boolean = true;
isNightTime ? console.log(...data: 'Turn on the lights!') : console.log(...data: 'Turn off the lights!');
```

Conditional Statements: falsy statements

❑ Falsy Statements are evaluated as **false** when checked as condition

- Empty strings like "" or "
- *null* which represent when there is no value at all
- *undefined* which represent when a declared variable lacks a value
- 0 or NaN (Not a Number)

❑ Any statement not evaluated as **false** will be evaluated as **true**

Conditional Statements: switch

- ❑ Evaluates an expression and executes code depending on the value of that expression.

```
switch (expressionToCheck) {  
    case value1:  
        // code executes if expressionToCheck matches value1  
        break;  
    case value2:  
        // code executes if expressionToCheck matches value2  
        break;  
    case value3:  
        // code executes if expressionToCheck matches value3  
        break;  
    default:  
        // code executes if variableToCheck does not match value1, value2 or value3  
        break;  
}
```

Conditional Statements: switch

- ❑ It can contain any number of case clauses. If the expression value matches the case value the block of code inside this case will be executed.
- ❑ A break at the end will indicate to exit the block preventing any more code to be executed or any other cases to be checked. further case clauses to be checked and executed.
- ❑ The default code will be executed if the expression doesn't match the value of any case.

Loops: for

- ❑ The for loop has three parameters separated by ;
 - Initialization of the iterator variable
 - Stopping condition (when the expression evaluates to **false**)
 - Iteration statement, updates the iterator for each loop

```
for (let i = 0; i < 10; i++) {  
    console.log(i);  
}
```

Loops: for...of and for...in

❑ The for...of iterates over an array.

```
const devices = ['android', 'laptop', 'tablet'];

for (const device of devices) {
  console.log(device);
}
```

❑ The for...in iterates over an object

```
const shoppingCart = { banana: 2, apple: 5};

for (const fruit in shoppingCart) {
  console.log(fruit);
  console.log(shoppingCart[fruit]);
}
```

Loops: while

- ❑ The `while` statement loops as long as a condition evaluates to **true**.
- ❑ The condition is specified before the loop
- ❑ Usually, a variable is altered inside the while loop code to determine when the loop must finish.

```
let i = 0;

while (i < 5) {
  console.log(i);
  i++;
}
```

Loops: do...while

- ❑ In this case, the loop executes a block of code and then checks if a condition is **true**. The loop is repeated until the condition evaluates to **false**.

```
let x = 0;
let i = 0;

do {
  x = x + i;
  console.log(x);
  i++;
} while (i < 5);
```


Hands-On Exercises:

- ☐ Log the string 'JavaScript' to the console
- ☐ Log the number 2024 to the console
- ☐ Log 'I love to code! #lasalle' to the console
- ☐ Log the float 19.54 to the console

Hands-On Exercises:

- ☐ Log how old a family member was when you were born (by subtracting your current age to their current age)
- ☐ Explain, what is the difference between printing $3 + 4$ and `'3 + 4'`?
- ☐ Log the two strings `'Hello'` and `'World'` concatenated.

Hands-On Exercises:

- ☐ Now log the strings 'Hello' and 'World', but this time make sure to also include a space (' ') between the two words.
- ☐ Log how many chars has the string 'Hello World'.
- ☐ Log the string 'Hello World' in all capital letters (using a method).

Hands-On Exercises:

- ☐ Initialise a variable 'num' and assign a random value between 0 and 100.
- ☐ Log if num is an integer (without using conditional statements; result should be a boolean).
- ☐ Ensure num to be an integer and log if num is bigger than 50 (without using conditional statements; result should be a boolean).

Hands-On Exercises:

- ❑ If num is smaller than 50, log the string 'num is smaller than 50'.
- ❑ Multiply num, using an assignment operator, by 5.
- ❑ If num is bigger than 50, log the string 'num is bigger than 50', if is equal to 50 log 'num is 50'. Otherwise log 'num is smaller than 50'.

Hands-On Exercises:

- ❑ Using a do...while log the first ten elements of the Fibonacci sequence